

Federated Intrusion Detection with Key-Value Cached Transformers in 5G RANs

Andrea Di Matteo^{1,2}, Emilio Paolini¹, Luca Valcarenghi¹, and Nicola Andriolli²

¹*Scuola Superiore Sant'Anna, Pisa, Italy*

²*Department of Information Engineering (DII), University of Pisa, Pisa, Italy*
andrea.dimatteo@santannapisa.it

Abstract—5G Radio Access Networks (RANs) are increasingly populated with heterogeneous devices that generate high-rate traffic streams, demanding security functions that operate close to the traffic source under strict latency and privacy constraints. However, most Network Intrusion Detection Systems (NIDSs) rely on centralized training and architectures such as CNN-based or shallow neural models, which (i) introduce bandwidth and latency overhead due to traffic aggregation, and (ii) struggle to capture long-range dependencies in attack patterns that unfold across multiple packets or flows. To address these issues, this paper proposes a packet-level Federated Learning framework for decentralized NIDS in 5G RANs, where network flows are modeled as sequences of packet tokens and processed using a Transformer-based architecture. The framework enables gNB-level collaborative training via FedAvg without sharing raw traffic, and supports real-time operation through an incremental Key-Value (KV) cache mechanism that reduces the computational cost of updating flow classifications as packets arrive. Experiments in an Industrial Internet of Things scenario exploiting the Edge-IIoTset dataset show that the transformer keeps an accuracy of about 94% across long packet sequences, consistently outperforming a CNN baseline. Federated training preserves most of the centralized performance while eliminating raw-traffic transfer, and KV-cached inference achieves ≈ 1 ms per-packet latency (up to $90\times$ speedup over standard self-attention), compatible with edge-level deployment in 5G RAN environments.

Index Terms—Federated learning, Network Intrusion Detection, Wireless Networks, Machine Learning, Transformers, Deep Learning, DDoS.

I. INTRODUCTION

5G networks have transformed the wireless telecommunications landscape, enabling unprecedented data volumes and speeds, ultra-low latency communications, and massive device connectivity [1], facilitating the adoption of Industrial Internet of Things (IIoT). These systems increasingly rely on software-defined, virtualized, and cloud-native infrastructures, where computation is distributed across the Radio Access Network (RAN), edge computing platforms, and the core network [2]. While this paradigm enables unprecedented flexibility and scalability, it also expands the network attack surface and introduces new security risks, particularly in environments where heterogeneous IIoT devices generate continuous high-rate traffic streams [3].

This work is supported by the Chips Joint Undertaking (JU), European Union (EU) HORIZON-JU-IA, under grant agreement No. 101140087 (SMARTY). It is also supported by the Italian MUR under the FoReLab Project (Departments of Excellence).

In this context, Network Intrusion Detection Systems (NIDSs) must evolve from centralized, offline monitoring tools into distributed, low-latency security functions operating close to the traffic source [4]. Security solutions [?], which aggregate network traffic at centralized servers or core-network monitoring points, suffer from three key limitations. First, they introduce additional latency and bandwidth overhead, which conflicts with the stringent real-time requirements of 5G services. Second, they expose sensitive user and industrial traffic to privacy risks, as raw packets must be transferred and stored in centralized repositories. Finally, many modern attacks progress gradually across multiple packets or flows, such as slowloris, stealth port scanning, or multi-stage intrusions, where malicious behavior becomes observable only over long temporal horizons; in centralized approach, this delayed and aggregated view of traffic makes such patterns harder to detect [5].

Additionally, many ML- and DL-based NIDS approaches rely on models that primarily capture local or short-range correlations, e.g., Convolutional Neural Networks (CNNs), in the traffic features [6]. While these models enable low latency inference due to their shared weights and local receptive filters, they struggle with long-range spatial and temporal patterns typical of modern distributed attacks (e.g., slowloris or multi-stage intrusions). As a consequence, packet-level correlations degrade rapidly as the context grows, and performance drops significantly when sequences exceed approximately ten packets [6].

Hence, to address the aforementioned challenges, we propose a decentralized packet-level NIDS framework designed for integration within 5G infrastructure and edge-IIoT environments. The framework is deployed at the user-plane level of the gNB and supports distributed training via Federated Learning (FL), enabling multiple network nodes to collaboratively train a global model without sharing raw traffic. Network flows are modeled as sequences of packet tokens, and we leverage a Transformer-based architecture to capture long-range dependencies across packets. To ensure real-time suitability, we further integrate a Key-Value (KV) cache mechanism to enable incremental flow classification as new packets arrive, reducing the computational overhead of self-attention from quadratic to approximately linear complexity. Experimental results on the Edge-IIoTset dataset demonstrate that the proposed framework achieves high detection accuracy

while preserving decentralization, maintaining performance under federated training, and providing bounded inference latency compatible with 5G edge environments.

II. RELATED WORK

The growing complexity of network traffic in 5G wireless networks has driven extensive research into intelligent edge NIDSs capable of operating under stringent real-time constraints.

Early research on 5G security mainly addressed protocol-level vulnerabilities, focusing on physical-layer security, lightweight encryption, and network slice isolation [7]. These techniques operate at the application and hardware layers and are incapable of recognizing rapidly evolving and complex traffic patterns typical of virtualized environments, such as 5G RANs. To overcome these limitations, research shifted towards data-driven approaches. Classical ML algorithms such as Decision Trees, Support Vector Machines (SVMs), and ensemble methods were employed to automate feature extraction [8]. While these models represented a significant advance over signature-based systems like Snort [9], they still required manual feature engineering and struggled to generalize across high-dimensional data typical of cloud-native infrastructures [10], [11].

To meet latency and throughput requirements of 5G networks, recent research has adopted Computer Vision (CV) techniques, encoding network flows as images to enable the use of CNNs [?]. These representations capture time and spatial correlations among packets using 2D convolutions, allowing lightweight CNNs to achieve high F1 scores with sub-millisecond inference times. However, the effectiveness of this approach degrades significantly on sequences exceeding 10 packets. As the input dimensions grow, the spatial correlation between packets vanishes rapidly, and CNNs struggle to capture the long-range dependencies due to local receptive filters.

Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks were introduced to handle sequential data, but they suffer from vanishing gradient and are difficult to parallelize, making them unsuitable for real-time deployment [12].

Transformer-based architectures have emerged as a dominant paradigm, originally in natural language processing [13], the self-attention mechanism enables the modeling of complex of long-range dependencies in sequential streams without the bottleneck of RNNs. FlowTransformer [14] provides a systematic analysis of architectural choices, demonstrating that modular head design critically affects detection accuracy. Hybrid pipelines, such as those combining rule-based engines with attention mechanisms, have achieved accuracy gains without compromising throughput [15].

Despite these advancements, most NIDSs rely on centralized training [16], [17]. This approach is becoming increasingly unfeasible due to privacy regulations and the prohibitive volume of data generated at the edge. FL offers a decentralized alternative, allowing models to be trained across distributed

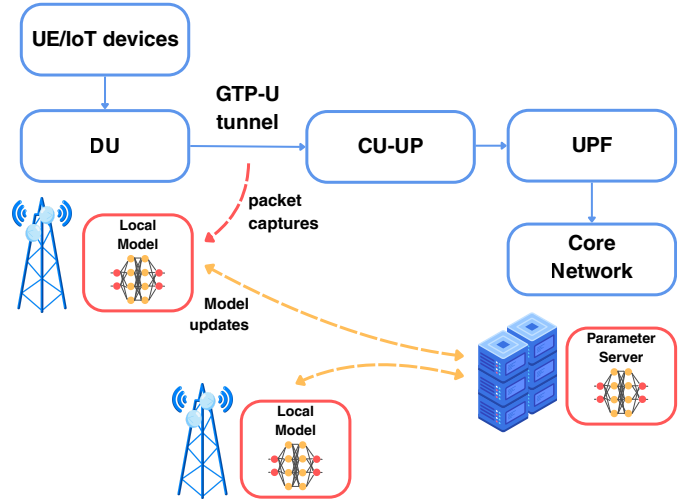


Fig. 1: Architecture of the FL based NIDS. Each client performs training on a local model with its own data. A central server deployed at the core network periodically aggregates model updates via FedAvg and sends back the updated model.

nodes without sharing raw data [18]. However, applying FL effectively to Transformer-based NIDS in wireless networks remains an open challenge, since current work focuses on simple Multi-Layer Perceptrons (MLPs) or CNNs in FL settings [19].

This work addresses this gap by combining packet-level feature extraction, Self-Attention based architectures with KV-caching for low latencies, and federated training to achieve a scalable, privacy-preserving, and real-time NIDS solution suitable for 5G RAN deployment.

III. FRAMEWORK AND DESIGN PRINCIPLES

The proposed framework is designed to integrate seamlessly within existing 5G infrastructure, ensuring standard compliance while enabling real-time, edge-level threat detection. Fig. 1 illustrates the end-to-end architecture for packet capture, local model training, and federated aggregation. The NIDS is deployed at the User Plane level and interfaces directly with the 5G data path, integrating with existing network functions without modifying the forwarding pipeline.

Packet extraction and preprocessing are performed at the GTP-U layer on the F1-U interface between the Distributed Unit (DU) and the Central Unit-User Plane (CU-UP), where user traffic is encapsulated and forwarded toward the core network. This positioning allows real-time inspection of packet- and flow-level statistics while preserving compliance with the 3GPP functional split and avoiding additional detours in the data path.

Each gNB hosts a local instance of the NIDS model, which performs inference on live traffic and periodically trains on locally collected packet traces. Only model updates (i.e., weights) are transmitted over the control plane to a Parameter Server (PS), while raw traffic never leaves the gNB. The PS is logically located in the core network domain, co-located with

MEC or orchestration infrastructure, and periodically aggregates the received updates using FedAvg before distributing the new global model to all participating nodes.

An alternative deployment strategy places the IDS instances alongside the UPF within the 5G Core. This approach is widely adopted, since the UPF acts as the enforcement and traffic steering point defined by PCFP rules [20]. However, from a performance perspective, this design introduces additional latency, as packets must traverse the CU-UP before being inspected, and it removes the possibility of exploiting fine-grained edge-level context. In contrast, our gNB-side deployment enables earlier detection, reduces raw-traffic transport overhead, and naturally supports FL by keeping data local.

A. Flow Construction

Raw packets captured at the gNBs are grouped into flows based on their 5-tuple (Source IP, Destination IP, Source Port, Destination Port, Protocol). For each flow, packets are sorted by increasing timestamp. We then extract a feature vector $x_i \in \mathbb{R}^F$ for each packet i . To prevent the model from memorizing endpoints, we exclude IP addresses and port numbers and the selected features include:

- Timing: Inter-arrival time (IAT) between current and previous packet.
- Size: Payload length and Header length.
- Flags: TCP window size, TCP flags (SYN, ACK, FIN, RST, PSH, URG), and TTL.
- Direction: Forward or backward indicator relative to the flow initiator.

Formally, a flow is represented as a matrix $X \in \mathbb{R}^{N \times F}$, where N is the context window size (number of packets) and F is the number of features. Flows shorter than N are zero-padded, and all features are min-max normalized, the overall representation is depicted in Figure 2.

While the CNN processes matrices as a single static images, the Transformer handles the flow sequentially. When a new packet x_t is captured at time t , it is appended to the sequence, and the oldest packet x_{t-N} is dropped in a First-In-First-Out fashion. This incremental processing structure motivates the use of a KV-cache, allowing the model to process only the new token x_t without recomputing attention for the entire history.

B. Model Architecture

Traditional CNN model treats the flow matrix X as a single $N \times F$ image [?]. It consists of a stack of 2D convolutional filters with skip connections, the output is then linearized to a single vector z followed by a fully connected Multi Layer Perceptron (MLP) to perform traffic classification. However, this approach primarily captures local spatial correlations within small neighborhoods of packets. As the flow length increases, relevant interactions between temporally distant packets are progressively attenuated, limiting the model's ability to detect slow-evolving or long-range behaviors within a sequence.

The Transformer model instead processes the entire sequence X jointly, allowing every packet token to attend to all

Flow: (SRC_IP, DST_IP, SRC_PORT, DST_PORT, PROTOCOL)

	Feature 1	Feature 2	Feature 3	...	Feature f
Packet 1	*	*	*	*	*
Packet 2	*	*	*	0	0
...	...	...	...	...	...
Packet N	*	*	*	*	*
...	...	...	...	...	...
Packet L	0	0	0	0	0

} Flow Matrix

Fig. 2: Packets are grouped in flows, identified by the 5-tuple src/dst ip, src/dst port and protocol, ordered by time and padded/truncated to N packets $\times f$ features. If the total flow is not a multiple of N then the matrix is padded with zeros as depicted in last row of the flow.

others within the flow. Each packet x_i is first projected into a latent space of dimension d_{model} to obtain its embedding, which is then combined with sinusoidal positional encodings to preserve temporal ordering. Through the self-attention mechanism, the model learns pairwise relationships between packets regardless of their distance in the sequence, enabling the representation of long-range temporal dependencies that are difficult to capture with locality-biased architectures.

The core component is the Multi-Head Attention (MHA) [21]. For a single head h , we compute Queries (Q_h), Keys (K_h), and Values (V_h) via linear projections:

$$\text{Head}_h = \text{softmax} \left(\frac{Q_h K_h^T}{\sqrt{d_k}} \right) V_h \quad (1)$$

where $d_k = d_{\text{model}}/H$ is the dimension per head. The outputs of all H heads are concatenated and projected:

$$\text{MHA}(X) = \text{Concat}(\text{Head}_1, \dots, \text{Head}_H) W^O \quad (2)$$

This mechanism allows every packet to be related to every other packet in the flow. Each encoder block also includes a Feed-Forward Network (FFN) and Layer Normalization [22]:

$$Z_{\text{out}} = \text{LayerNorm}(Z_{\text{in}} + \text{FFN}(Z_{\text{in}})) \quad (3)$$

The output is then linearized to a single vector z followed by a fully connected MLP as classification head.

C. Key-Value cache

Standard Transformer architecture requires re-computing attention of tokens at each step by its autoregressive property. Let d be the dimension of the embeddings and N the sequence length, i.e., our context window size.

In the standard approach, when the t -th packet x_t arrives, the model computes the attention for the entire shifted sequence $x_{t-N}, \dots, x_t$. The Query, Key, and Value matrices are $Q_t, K_t, V_t \in \mathbb{R}^{N \times d}$. The computational bottleneck is the

matrix multiplication $Q_t K_t^T$, which requires $N^2 \cdot d$ operations (ignoring softmax and scaling operations). Thus, the complexity for processing the single t -th packet is $O(N^2)$, making the standard self-attention formulation impractical for latency-sensitive NIDS scenarios.

To address this limitation, we introduce a Key-Value (KV) cache to enable incremental inference, represented in Fig. 3. Assuming that the cache already stores the Key and Value matrices computed for the previous tokens, K_{t-1} and V_{t-1} , when the token x_t arrives at time t , only the representations associated with the new packet are computed and appended to the cache:

- 1) We compute the query, key, and value vectors only for the new token: $q_t, k_t, v_t \in \mathbb{R}^{1 \times d}$.
- 2) We append k_t and v_t to the cached matrices:

$$K_{cache} \leftarrow [K_{cache}; k_t], \quad V_{cache} \leftarrow [V_{cache}; v_t]$$

where $K_{cache}, V_{cache} \in \mathbb{R}^{N \times d}$.

- 3) Self attention is computed using only the new query q_t against the full cache:

$$\text{Attn}_t = \text{softmax} \left(\frac{q_t K_{cache}^T}{\sqrt{d}} \right) V_{cache}$$

The matrix multiplication $q_t K_{cache}^T$ involves a vector ($1 \times N$) and a matrix ($d \times N$), resulting in $N \cdot d$ operations. By assuming that the embedding dimension $d \ll N$, the complexity for processing the t -th packet reduces to $O(N)$, thus ensuring the system remains efficient enough for low latency constraints.

Moreover, although the computational complexity of incremental attention is linear in the context length, the massive parallelism of modern GPUs allows the vector-matrix operations to be executed concurrently across multiple processing threads. As a result, the wall-clock inference latency increases only marginally with N and remains effectively bounded within the latency budget required for real-time NIDS operation.

D. Federated Learning

To enable distributed training across the gNBs we exploit FL into the proposed architecture. Each base station hosts a local instance of the Transformer model and performs both inference and local training using observed traffic flows. We consider a network consisting of K distributed clients, i.e., the gNBs. Each client possesses a local dataset of packet captures in .pcapng $\mathcal{D}_k$ of network flows; the goal is to train a global model w at the parameter server that minimizes the loss function $F(w)$:

$$\min_w F(w) = \sum_{k=1}^K \frac{n_k}{N} F_k(w) \quad (4)$$

where $N = \sum_k n_k$ is the total number of samples, and $F_k(w)$ is the local loss function at client k , namely the Cross-Entropy loss over the local data distributions $(x, y) \sim \mathcal{D}_k$. Each gNB sends to the parameter server metadata such as the number of local samples along with the local model weights w^k .

$$F_k(w) = \mathbb{E}_{(x,y) \sim \mathcal{D}_k} [\ell(f_w(x), y)] \quad (5)$$

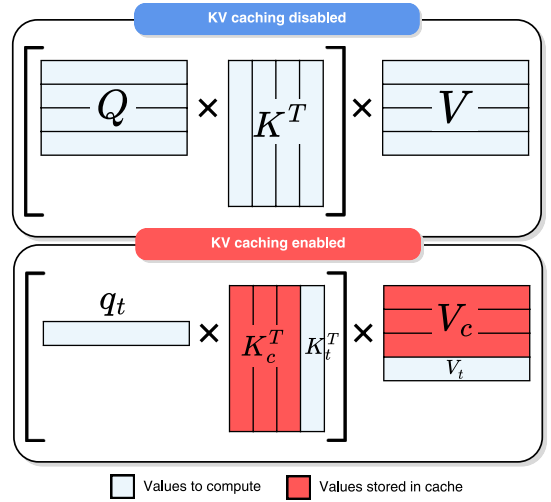


Fig. 3: Visual representation of key-value caching mechanism. The K_c and V_c are the cached keys and values. When a new packet q_t arrives at time t only the respective K_t and V_t are computed and finally added to the cache. When the cache saturates to the context window size, the oldest vector is dropped.

For simplicity we assume Independent and Identically Distributed (IID) data distribution among the gNBs and we refer to standard FedAvg algorithm [23]. In each communication round t :

- 1) The parameter server broadcasts the current global model w_t to a subset of eligible gNBs.
- 2) Each federated client k performs local training via Stochastic Gradient Descent for E epochs to obtain an updated model w_{t+1}^k :

$$w_{t+1}^k \leftarrow w_t - \eta \nabla F_k(w_t) \quad (6)$$

where η is the learning rate.

- 3) Federated clients send their updates $\Delta w^k = w_{t+1}^k - w_t$ (or the weights directly) back to the server.
- 4) The server aggregates the updates to produce the new global model:

$$w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{N} w_{t+1}^k \quad (7)$$

This design limits bandwidth usage to model weights and metadata of negligible size, preserving the privacy of raw traffic data.

Importantly, the IID assumption may not hold in practice, as different gNBs serve heterogeneous traffic profiles. In such cases, FedProx-style proximal regularization [24] can be adopted to bound client drift during local training.

IV. EXPERIMENTS AND RESULTS

To evaluate the proposed approach, we compare the detection performance of the Transformer architecture with KV caching against a CNN baseline, analyze its behavior under

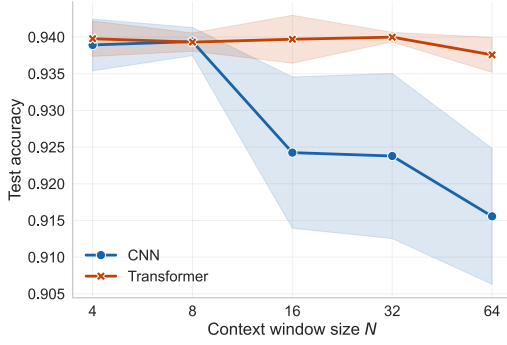


Fig. 4: Accuracy vs. context window size N . Shades represent the Confidence Interval (CI) at 95% confidence level.

FL training, and assess its real-time inference latency with and without KV caching.

A. Experimental Setup

We use the Edge-IIoTset dataset, which contains IIoT traffic with benign packets and 14 distinct attack classes, totaling approximately 72 million packets [25]. The packet features used (IAT, payload size, TCP flags, direction) are standard protocol-level statistics, making the framework applicable beyond IIoT to general 5G deployments. Following packet processing, the data is divided into train/validation/test sets with 70/20/10 split. For FL experiments, the training data is partitioned across clients using an IID sampling.

The CNN uses 4 layers of 2D convolutions. The Transformer uses 4 encoder layers with 4 attention heads each. The classification head consists of a global average pooling layer followed by a dense layer. Training uses the Adam optimizer ($lr = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$) in both cases. Experiments are conducted on a server with dual Intel Xeon Gold 6244 CPUs and an NVIDIA Tesla V100S GPU. Latency benchmarks are averaged over 50 runs with different seeds to ensure statistical relevance.

B. Transformer and CNN comparison

We first analyze the detection accuracy as the context window N increases. The results, plotted in Fig. 4, indicate a different trend between the two models. The CNN baseline achieves high accuracy for short context windows, reaching 0.939 at $N = 4$, but its performance gradually degrades as the window increases, dropping to 0.916 at $N = 64$.

This degradation suggests that CNNs saturate on local patterns. As the context window increases, relevant attack patterns (e.g., the specific sequence of flags in a port scan) may be separated by long sequences of benign packets. The limited local receptive filters mechanism of CNNs prevents it from correlating these distant features. Conversely, the Transformer maintains a stable accuracy across all context window sizes, remaining close to 0.94 (e.g., 0.938 at $N = 64$). The self-attention mechanism effectively allows the model to

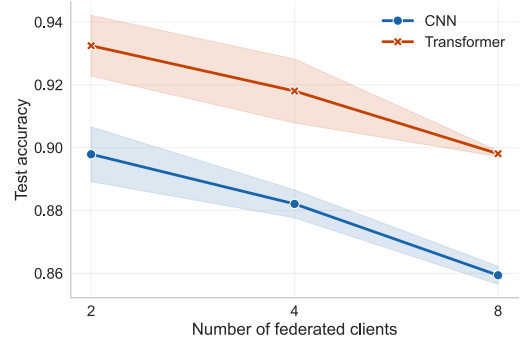


Fig. 5: Accuracy vs. number of clients in a FL setting. Shaded areas represent the CI at 95% confidence level.

pay attention to the entire flow history regardless of their temporal distance. This confirms the hypothesis that long-range dependency modeling is crucial for robust NIDS, especially for complex attacks that intentionally space out malicious packets.

C. Federated Learning Analysis

We evaluate the impact of distributing the training data across $K \in \{1, 2, 4, 8\}$ clients while maintaining a constant total training budget. We limit K to 8 to reflect a realistic scale for gNB-level FL deployments. To ensure a fair comparison across different values of K , we fix the number of FL rounds to $R_{FL} = 25$ and keep the cumulative number of epochs across all clients constant at 200 ($E_{total} = K \times E_{local} \times R_{FL} = 200$). The overall results of training are depicted in Fig. 5.

As the number of federated clients K increases, each client trains on a progressively smaller local dataset of size ($n_k = N/K$), reducing the diversity of traffic patterns available locally and ultimately limiting the quality of the learned representations. This effect is visible in both architectures: the CNN drops from approximately 0.899 accuracy at $K = 2$ to 0.860 at $K = 8$, while the Transformer exhibits a similar trend, decreasing from about 0.933 at $K = 2$ to 0.899 at $K = 8$. Despite this shared degradation trend, the transformer consistently outperforms the CNN across all federated configurations, suggesting that the self-attention mechanism is more resilient to data fragmentation and client-level variability than the locality-biased CNN baseline.

D. Latency Analysis and Optimization

Since NIDS deployed at the 5G edge must operate under strict latency constraints, we evaluate the inference latency of the CNN model and the Transformer architecture, with and without KV caching. Results are reported in Fig. 6.

The CNN remains the fastest architecture, demonstrating extremely stable sub-millisecond inference times (≈ 0.40 ms) regardless of the sequence length, thanks to the efficiency of 2D convolution operations on modern hardware. Standard Transformer architecture instead scales quadratically ($O(N^2)$) with the sequence length N . Its computational cost becomes prohibitive for long flows, rising from 0.90 ms for $N = 2^4$

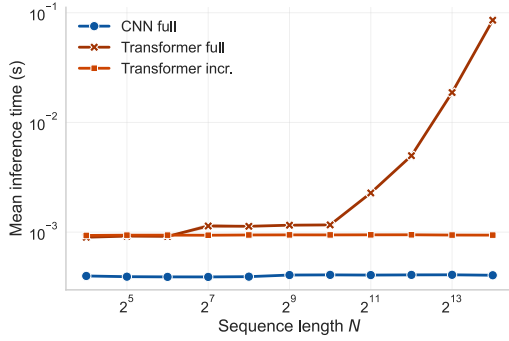


Fig. 6: Mean inference time comparison.

to over 85 ms at $N = 2^{14}$. The KV-cache mechanism successfully flattens the complexity per token. While it does not offer any speedup at $N = 16$, it provides a massive $\sim 91\times$ speedup at $N = 2^{14}$, keeping inference times below 1 ms. Experimentally, the incremental inference time appears nearly constant regardless of flow length even if we proved this to be approximately linear. This is due to the massive parallelism of modern GPU hardware, which processes the attention scores concurrently.

Although the CNN remains the fastest model in absolute terms, the Transformer with incremental inference fits within typical 5G RAN latency constraints, offering superior accuracy and robustness for complex patterns at a slightly higher, but bounded, computational cost.

V. CONCLUSION

This work proposed a comprehensive framework for decentralized NIDS in 5G/IIoT networks, leveraging FL and transformer architectures with KV caching for achieving higher performances.

We showed that standard CNNs suffer from accuracy degradation as the flow context increases, whereas the Transformer effectively captures long-range dependencies through self-attention and maintains a high and stable accuracy of approximately 0.94 across all evaluated window sizes). Through the implementation of a Key-Value cache, we addressed the quadratic computational cost of Transformers, reducing the complexity of incremental inference from quadratic to approximately linear and enabling sub-millisecond per-packet latency across all evaluated sequence lengths. Furthermore, the FL experiments confirmed that the system preserves most of the centralized accuracy (e.g., ≈ 0.93 at $K = 2$ and ≈ 0.90 at $K = 8$) while eliminating raw-traffic transfer and therefore reducing both privacy exposure and bandwidth overhead.

Future work will address non-IID distributions via FedProx, explore semi-supervised pre-training of the Transformer encoder on unlabeled edge traffic, and validate the framework on additional 5G deployment scenarios beyond IIoT.

REFERENCES

- [1] X. Lin, "An overview of 5G advanced evolution in 3GPP release 18," *IEEE Commun. Standards Mag.*, vol. 6, no. 3, pp. 77–83, 2022.
- [2] T. Taleb *et al.*, "On multi-access edge computing: A survey of the emerging 5G network edge cloud architecture and orchestration," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1657–1681, 2017.
- [3] M. Wazid *et al.*, "Security in 5G-enabled internet of things communication: issues, challenges, and future research roadmap," *IEEE Access*, vol. 9, pp. 4466–4489, 2020.
- [4] R. Chinnnasamy *et al.*, "Deep learning-driven methods for network-based intrusion detection systems: A systematic review," *ICT Express*, vol. 11, no. 1, pp. 181–215, 2025.
- [5] M. Ring *et al.*, "Detection of slow port scans in flow-based network traffic," *PLoS one*, vol. 13, no. 9, p. e0204507, 2018.
- [6] E. Paolini *et al.*, "Real-Time Clustering Based on Deep Embeddings for Threat Detection in 6G Networks," *IEEE Access*, vol. 11, pp. 115 827–115 835, 2023.
- [7] X. Ji *et al.*, "Overview of 5G security technology," *Science China Information Sciences*, vol. 61, no. 8, p. 081301, 2018.
- [8] Z. Xu *et al.*, "Robust anomaly detection in network traffic: Evaluating machine learning models on CICIDS2017," *arXiv preprint arXiv:2506.19877*, 2025.
- [9] M. Roesch, "Snort: Lightweight intrusion detection for networks," in *Proc. 13th USENIX Sys. Admin. Conf. (LISA)*, 1999, pp. 229–238.
- [10] M. A. Shyaa *et al.*, "Evolving cybersecurity frontiers: A comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems," *Engineering Appl. of Artif. Intell.*, vol. 137, p. 109143, 2024.
- [11] M. A. Ferrag *et al.*, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *Journal of Information Security and Applications*, vol. 50, p. 102419, 2020.
- [12] S. Hochreiter *et al.*, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [13] A. Zeng *et al.*, "Are transformers effective for time series forecasting?" *Proc. AAAI Conf. on Artif. Intell.*, vol. 37, no. 9, pp. 11 121–11 128, 2023.
- [14] L. D. Manocchio *et al.*, "Flowtransformer: A transformer framework for flow-based network intrusion detection systems," *Expert Systems with Applications*, vol. 241, p. 122564, 2024.
- [15] K. Changrampadi *et al.*, "Snort meets transformers: Accelerating transformer-based network traffic classification for real-time performance," in *Proc. 18th European Workshop on Systems Security (EuroSec '25)*. ACM, 2025, pp. 33–39.
- [16] M. Issa *et al.*, "Systematic literature review on intrusion detection systems: Research trends, algorithms, methods, datasets, and limitations," *Journal of Intelligent Systems*, vol. 33, 2024.
- [17] A. Bendale *et al.*, "Towards open set deep networks," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1563–1572.
- [18] P. Kairouz *et al.*, "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1–2, p. 1–210, Jun. 2021.
- [19] K. Bonawitz *et al.*, "Towards federated learning at scale: System design," in *Proc. 2nd SysML Conf.*, vol. 1, 2019, pp. 374–388.
- [20] "3GPP TS 29.244: Packet Forwarding Control Protocol (PFCP)," 3rd Generation Partnership Project (3GPP), Tech. Rep. V16.5.0, 2020, release 16.
- [21] A. Vaswani *et al.*, "Attention is all you need," in *Proc. 31st Internat. Conf. on Neural Information Processing Systems*, 2017, p. 6000–6010.
- [22] J. L. Ba *et al.*, "Layer normalization," 2016. [Online]. Available: <https://arxiv.org/abs/1607.06450>
- [23] B. McMahan *et al.*, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. 20th Conf. on Artificial Intelligence and Statistics*. PMLR, 2017, pp. 1273–1282.
- [24] T. Li *et al.*, "Federated optimization in heterogeneous networks," *Proceedings of Machine learning and systems*, vol. 2, pp. 429–450, 2020.
- [25] M. A. Ferrag *et al.*, "Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IIoT and IIoT Applications for Centralized and Federated Learning," *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022.